



Module 3 — Spring Data Access

A session designed for freshers with Core Java and SQL knowledge. The focus is on **understanding the architecture and flow** — not memorizing APIs.





Module Overview

This module covers the full Spring Data Access stack — from the problems with raw JDBC to modern JPA repositories.

Why Spring Data?

The motivation behind data access frameworks

Spring JDBC

JdbcTemplate, DAO & Repository patterns

Spring ORM

Hibernate basics and ORM concepts

Spring Data JPA

Ready-made repositories with minimal SQL



Why Do We Need Spring Data?

Every App Needs

- Save Data
- Read Data
- Update Data
- Delete Data

Without Spring

Java code becomes **repetitive and verbose**. Every operation requires:
Connect → Execute SQL → Read Result
→ Close Connection.

Spring simplifies **all** these steps into clean, concise method calls — reducing boilerplate dramatically.

Traditional JDBC — The Old Way

Before Spring, every database interaction required a rigid sequence of manual steps. This made even simple queries verbose and error-prone.

- 1

2

3

4

5

6
1. Load Driver

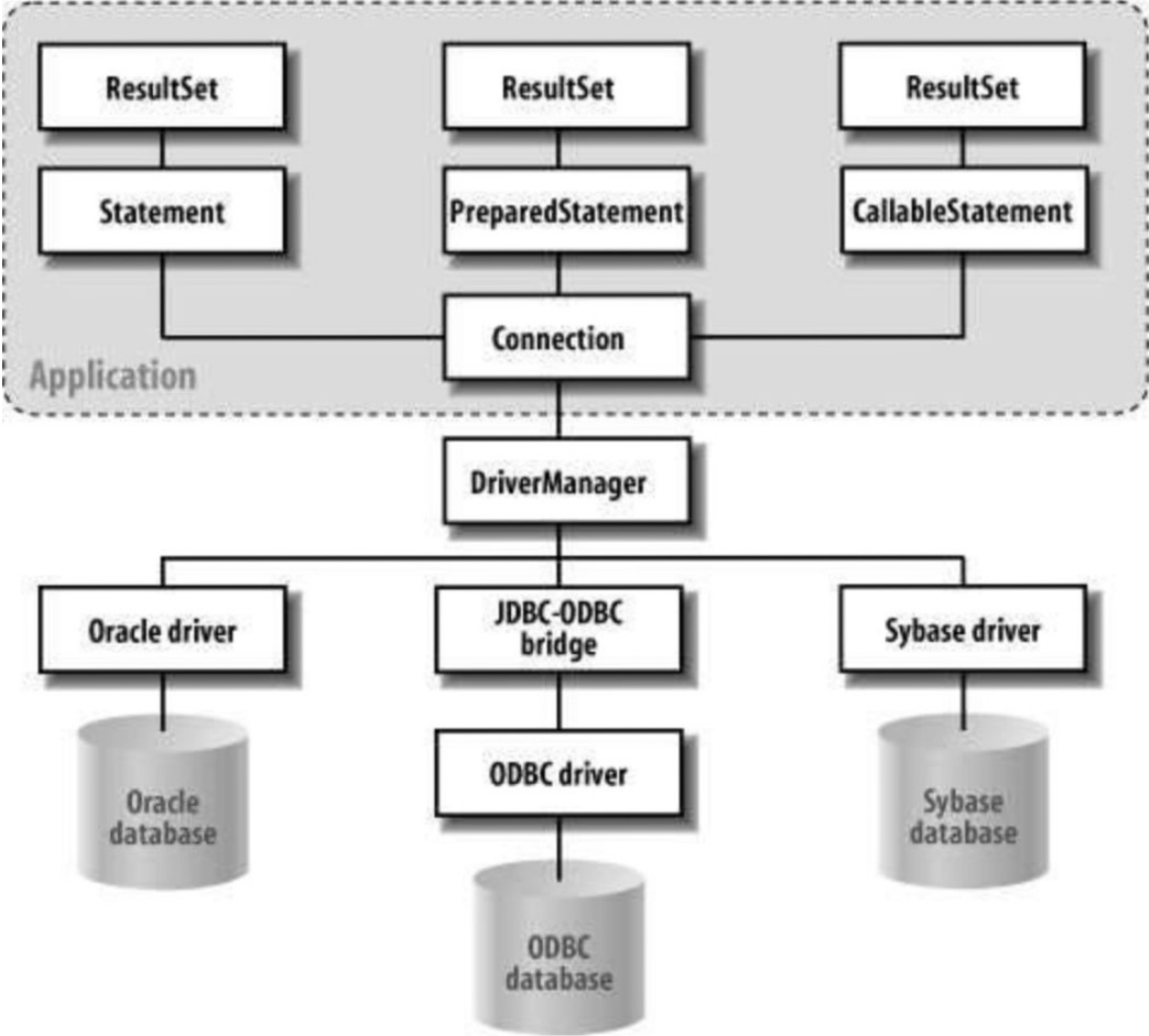
2. Open Connection

3. Create Statement

4. Execute SQL

5. Process ResultSet

6. Close Resources



Problems with Traditional JDBC

Too Much Code

Even a simple query requires dozens of lines of boilerplate.

Repeated Everywhere

The same connection and cleanup code is duplicated across every DAO method.

Manual Resource Closing

Forgetting to close connections causes memory leaks and bugs.

Complex Exception Handling

`try-catch-finally` blocks appear in every method, cluttering the logic.

Hard to Maintain

Transaction management is difficult and the code is fragile to change.

How Spring JDBC Solves It

Spring introduces **JdbcTemplate** — a powerful abstraction that eliminates boilerplate. Instead of managing connections, statements, and result sets manually, you simply call clean methods.

Before Spring

Connection → PreparedStatement → ResultSet → Close

Repeated in every method. Error-prone and verbose.

After Spring

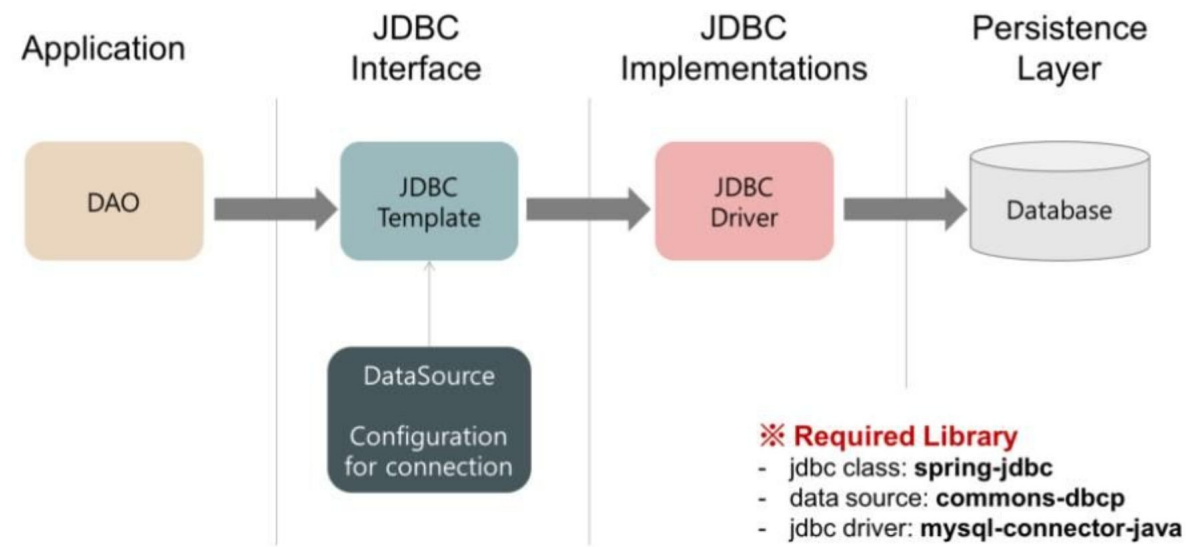
```
jdbcTemplate.query()
```

```
jdbcTemplate.update()
```

Spring handles connections, resource closing, and exceptions automatically.

Spring JDBC Architecture

Spring JDBC introduces a clean layered architecture. Each layer has a single responsibility, making the application easier to maintain and test.



-Data Access Layer-

Application Business Logic	Service Orchestration	DAO Database Operations
JdbcTemplate Spring Abstraction	Database Persistence	

JdbcTemplate — Core Methods

JdbcTemplate is the most commonly used class in Spring JDBC. It provides simple methods for all database operations — no manual connection management needed.



query()

Retrieve a list of rows from the database, mapped to Java objects.



update()

Execute INSERT, UPDATE, or DELETE statements.



queryForObject()

Retrieve a single row or scalar value.



batchUpdate()

Execute multiple SQL statements in a single batch for efficiency.

CRUD Using JdbcTemplate

Spring's JdbcTemplate converts standard SQL operations into simple, readable method calls. Each CRUD operation maps directly to a method.



Create

INSERT → jdbcTemplate.update()



Read

SELECT → jdbcTemplate.query()



Update

UPDATE → jdbcTemplate.update()



Delete

DELETE → jdbcTemplate.update()

The left side of the slide features a dark teal background with a grid pattern. Several 3D cubes, some containing code snippets, are floating in the air. At the bottom, there is a large, glowing grid of smaller squares, resembling a data table or a database structure.

RowMapper — Bridging Database and Java

The Problem

A database query returns **rows**. Java applications need **objects**. These two worlds need a translator.

The Solution

RowMapper converts each database row into a Java object automatically. Every row in the `Student` table becomes one `Student` object.

Flow

Student Table

↓ RowMapper ↓

Student Object

Simple, clean, and reusable across queries.

DAO Layer — Data Access Object

The **DAO pattern** separates database logic from business logic. A DAO class contains only the code needed to interact with the database — nothing more.

Responsibilities

- Only database logic
- No business logic
- Wraps JdbcTemplate calls

Example: StudentDAO

- `save()`
- `findById()`
- `findAll()`
- `delete()`

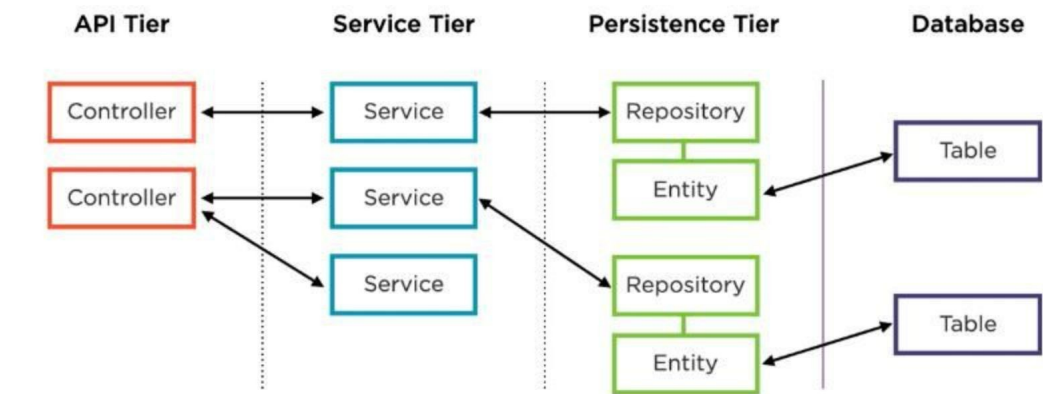
Benefits

- Reusable across the app
- Easy to maintain
- Simple to unit test

Repository Pattern — Modern Spring Architecture

Modern Spring applications use the **Repository pattern** instead of traditional DAO. The Repository hides SQL implementation details, resulting in a cleaner, more maintainable architecture.

Repository Architectural Overview



Controller	Service
Repository	Database

Spring JDBC vs. Repository Pattern

Both approaches have their place. Understanding when to use each is key to good architecture decisions.

Aspect	Spring JDBC	Repository
SQL Writing	Manual	Mostly automatic
Control	More control	Less code
Best For	Custom SQL queries	Standard CRUD
Complexity	Medium	Low

Introduction to ORM

What is ORM?

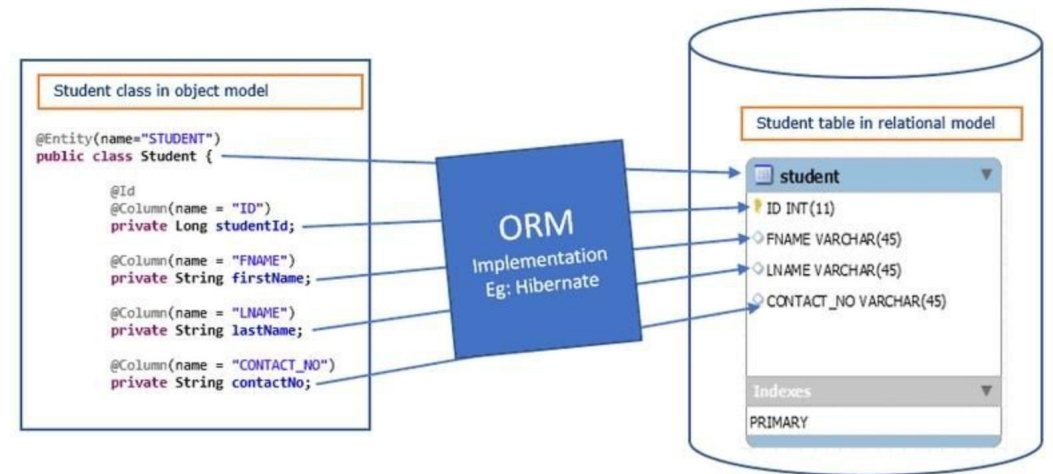
Object Relational Mapping — a technique that lets you work with Java objects instead of writing raw SQL.

Instead of writing:

```
INSERT INTO Student VALUES (...)
```

You work with:

A `Student` object — Spring converts it into the appropriate database row automatically.



ORM implements responsibility of mapping the Object to Relational Model.  JavaByDeveloper

A teal, three-dimensional cube is positioned on the left, with a large, stylized arrow emerging from its right side. The arrow is colored with a gradient from teal to orange and points towards a dark grey, cylindrical database icon on the right. The background is a warm, orange-brown gradient.

What is Hibernate?

Hibernate is the most popular Java ORM framework. Spring commonly uses Hibernate internally to manage database communication.

Maps Java Objects

Converts Java classes to database tables automatically.

Generates SQL

Writes INSERT, SELECT, UPDATE, DELETE behind the scenes.

Manages DB Communication

Handles sessions, transactions, and connections.

Advantages

Less SQL, cleaner code, and portable across databases.

Hibernate Core Concepts

For freshers, only the foundational concepts need to be introduced — no deep internals required at this stage.

Entity

A Java class (e.g., `Student`) that maps to a database table.
Annotated with `@Entity`.

Session

The main interface for database interaction — used to save, find, and delete objects.

Mapping

Defines how a Java class maps to a table: fields → columns, class → table name.

Persistence Context

Tracks the state of all managed objects during a session lifecycle.

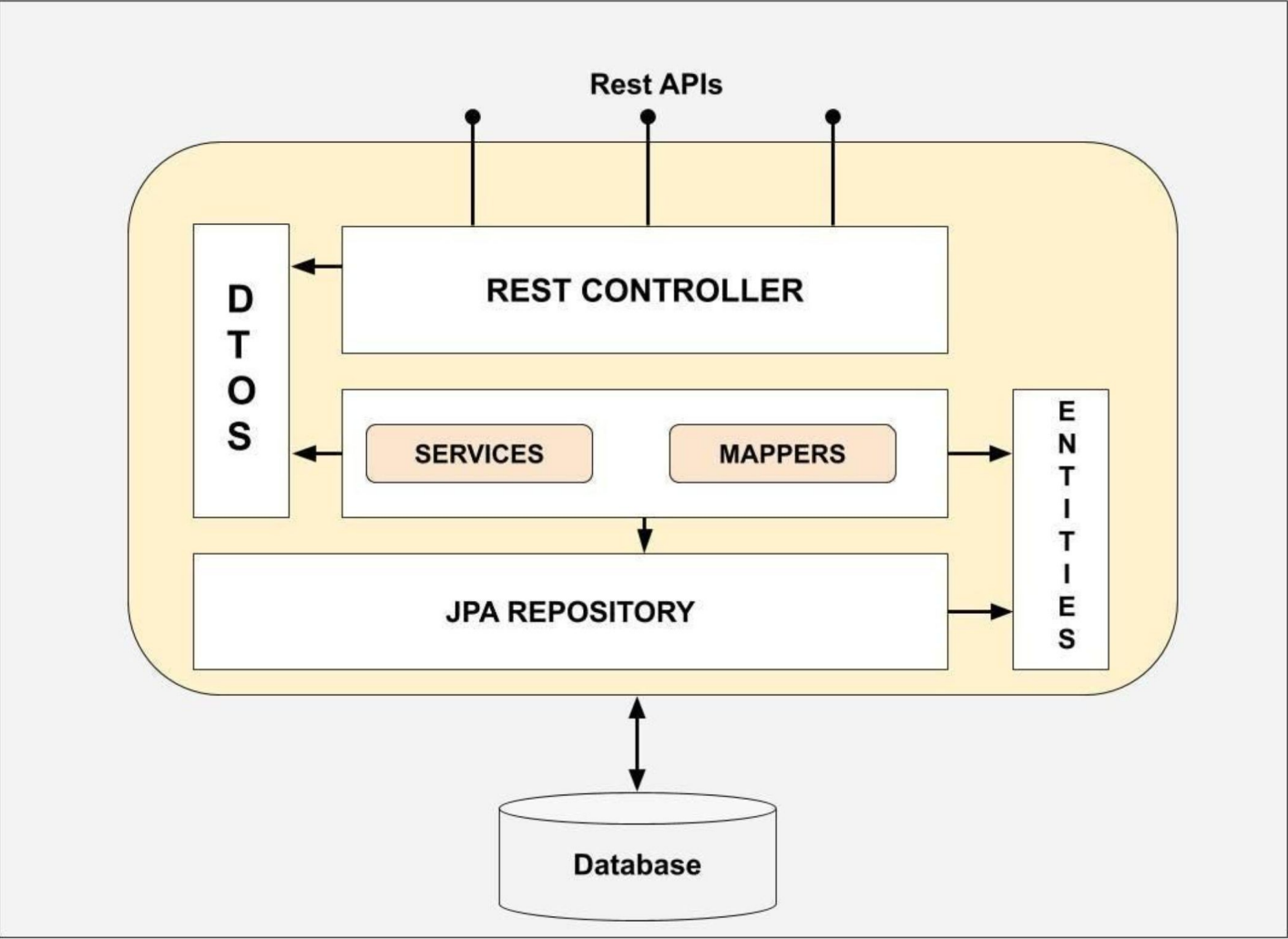
Spring ORM Architecture

Spring manages Hibernate's sessions, transactions, and repository dependencies — so developers focus on business logic, not infrastructure.



Spring Data JPA — Repositories Made Easy

Spring Data JPA provides **ready-made repositories** so developers write almost no SQL. Extending `JpaRepository` gives you all standard operations out of the box.



Save Persist a new or updated entity	Find Retrieve by ID or custom criteria
Delete Remove an entity from the database	Pagination & Sorting Built-in support for large datasets

Module Summary

Students completing this module should now have a solid conceptual understanding of the full Spring Data Access stack.

01

Traditional JDBC & Its Problems

Verbose, repetitive, and hard to maintain

02

Spring JDBC & JdbcTemplate

Simplified data access with clean method calls

03

RowMapper & DAO Pattern

Mapping rows to objects and separating concerns

04

Repository Pattern

Modern, cleaner architecture for data access

05

ORM, Hibernate & Spring Data JPA

Object-first development with minimal SQL